

Evolutionary programming and genetic algorithms

Course 1. Introduction

Evolutionary computation (EC)

- Informatics domain inspired by natural evolution process
- Natural evolution $\leftrightarrow$ solve problems as trial-and-error/generate-test
- Evolutionary computation refers to evolutionary programming, genetic algorithms and genetic programming, evolutionary strategies and swarm intelligence
- Limited resources environment
- Maximum number of individuals (phenotypes)
- Reproduction (crossover), occasional mutations
- Fitness, Competition, Selection

Evolutionary algorithms (EA) classification:

- Genetic algorithms – finite alphabet strings
- Evolutionary strategies – real number vectors
- Genetic programming – trees
- Evolutionary programming – finite state machines
- Swarm intelligence – real number vectors

Natural evolution $\leftrightarrow$ problem solving

Evolution (genetic approach)	Problem solving (real-world approach)
Environment (search space, solution space)	Problem
Individual (genotype, chromosome, candidate solution)	Candidate solution (phenotype)
Fitness (adaptation)	Quality (optimum)

Genotype = the internal representation of a solution / the “hidden code” or “blueprint” of something

Phenotype = the actual expression or interpretation of the genotype / what you actually see or use in real life

Example with a Video Game Character 🎮

- **Genotype:** A character's stats stored as numbers in the game code (e.g., `Strength = 10, Speed = 7, Health = 5`).
- **Phenotype:** How the character behaves in the game (e.g., runs fast, punches hard, but has low health).

Evolution (of population):

- Phenotype (x, y axis) -> evaluation, fitness (z axis)
- Population "advances" towards "higher" areas (better quality)
- Unimodal/multimodal problems
- Genetic inactivity, population concentration in local optima

Problem types:

classic model: inputs -> model -> outputs

1. Optimization problems

- Known: model, desired output
- Must compute: inputs
- Examples: TSP, job shop scheduling, N-queens problem

2. Modelling problems/system identification

- Known: inputs, outputs
- Must compute: model (with adequate generalization ability)
- Examples: supervised classification for 2 class model, machine learning, data mining

3. Simulation problems:

- Known: inputs, model
- Must compute: outputs
- Examples: "what-if?" questions, evolutionary economy

General scheme of EA

1. Candidates set: D

D is the "search space" = all possible solutions. Each candidate solution belongs to this space.

2. Fitness function $f:D \rightarrow R$, maximized/minimized

Individual $x \in D$, individual fitness (evaluation) $f(x)$

The fitness function evaluates the phenotype – when evaluating fitness, the genotype is converted into a phenotype to see how well it performs. It tells how „good“ a solution is.

3. Randomly choose a set of initial candidates $P_0 \subseteq D$ (initial population)

P_0 contains candidate solutions (genotypes).

4. Evaluate candidates $f(x)$, $x \in P_0$

Computing the fitness for each solution in the population.

5. Compute next generation

- Select „parents“, the „best“ individuals from current population $BP_0 \subseteq P_0$ – choose the best candidates (higher fitness) to act as parents.
- $BP_0 \rightarrow$ new individuals („children“) + $P_0 \rightarrow$ next generation P_1 (competition)

In order to generate new children, combine parents (using crossover) to create new solutions and add some randomness (mutation) to introduce diversity.

The next generation consists of some old and new individuals – the best ones survive and the process is repeated (competition).

- $P_0 \leftarrow P_i$, repeat from previous step until a sufficiently fit individual is obtained or a computational limit is reached
- 6. **Fundamental elements** → increase population quality
 - Variation operators: crossover, mutation ↔ maintains population diversity, avoids getting stuck
 - Selection operators ↔ keep the best solutions improving
 - Balance between exploration and exploitation ↔ exploring new solutions while improving old ones
- 7. **Operator directed search**
 - The algorithm gradually improves solutions step by step and the solutions adapt to the environment (the problem)
- 8. **Stochastic factor**
 - The initial population is chosen randomly.
 - Parent selection: chances proportional to quality
 - **Crossover**: select individuals and pairs (tuples) of parents for crossover, probability; select sequences of genetic material to be exchanged
 - **Mutation**: select individuals and genes, probability; select new values (alleles)

General algorithm of EA

1. $i = 0$ //begin with random solutions
2. **Initialize the population**. Build P_i by random selection //create the first generation, generate random individuals
3. **Evaluate* candidates**. For each $x \in P_i$ compute $f(x)$ //evaluate fitness – compute how good each solution is
4. **Repeat**.
 - 4.1. **Select** parents $BP_i \subseteq P_i$ //pick the best individuals
 - 4.2. **Recombine** pairs (or n-tuples) or parents → children (descendants) //crossover
 - 4.3. Apply **mutation** on children → new candidates //randomly modify some children to introduce diversity
 - 4.4. **Evaluate*** new candidates //compute fitness for the new individuals
 - 4.5. **Select** individuals for the next generation P_{i+1} //choose the best individuals for the next round
 - 4.6. $i \leftarrow i+1$

Until **termination condition** is met

*Evaluation might be needed immediately after creation of each individual

Crossover = when 2 parent solutions combine to create one or more children (new solutions).

(crossover combines traits from 2 parents and creates new solutions that inherit characteristics from both parents).

Mutation = small random change in a solution to introduce diversity and prevent premature convergence.

Example. Optimization of 1 variable function

Consider $f: [-1,1] \rightarrow \mathbb{R}$. $f(x) = x^3 - 3\sin(7x + 0.2) + 2\cos(x/5 - 0.4) + 1$. Find x for which $f(x)$ is maximum.

- **Phenotype space:** all real numbers in $[-1, 1]$
- **Genotype space:** all real numbers in $[-1, 1]$
- **Population size:** dim individuals, randomly chosen
- **Parent selection:** $dim/2$ individuals – select half of the current population using 2 member tournament method
- **Crossover operator:** $dim/4$ individuals – $x, y \rightarrow z = (x+y)/2$
- Crossover probability pr
- **Mutation operator:** $dim/4$ individuals – $x \rightarrow -x$
- Mutation probability pm
- **Next generation selection:** superelitist
- Best dim individuals from current population + new individuals
- Determinist mechanism
- **Termination condition:** maximum number of permitted generations (epochs) OR mean population quality does not significantly improve

Course 2. EA components and Hillclimbing algorithm

EA components:

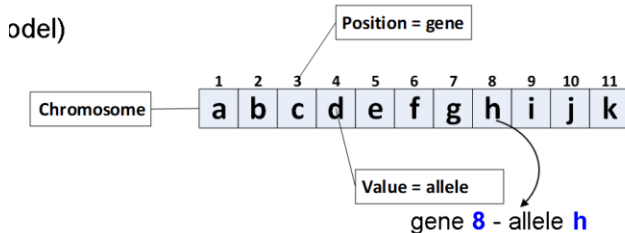
- Representation – chromosomes/individuals/population members
- Evaluation function (fitness)
- Population (sample)
- Initialization (initial population)
- Selection operators (mechanisms) $\rightarrow$ parents, survivors
- Variation operators $\rightarrow$ crossover (recombination), mutation
- Termination condition

Representation:

Relation real problem environment $\leftrightarrow$ problem solving space

- Encoding operations, data structures
- **Phenotype space** – solution space (real)
Phenotype = individual = candidate solution = point in the multi-dimensional solution space
- **Genotype space** – solution space (model)
Genotype = individual = chromosome
- **Phenotype <-> genotype**

$phenotype \xrightarrow{encoding} genotype \quad genotype \xrightarrow{decoding} phenotype$

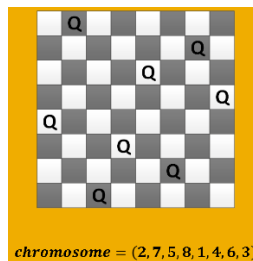


Gene = a position in the genotype (a characteristic of the solution).

Allele = the actual value the gene takes.

Evaluation function (quality/fitness):

- Assesses the adaptation of the individual
- **Main purpose:** selection processes, chose solution, terminate search
- **Evaluation function on phenotype space -> evaluation function on genotype space -> fitness**
- Optimum <-> maximum value -> f must be maximized
- Optimum <-> minimum value -> f or 1/f
- Best individual* <-> individual with



or $1/(c+f)$ must be maximized
maximum f value

The fitness function helps the algorithm (individuals) are better and should be It takes a candidate solution (genotype) and well it solves the problem. The algorithm either maximizes or minimizes this score.

decide which solutions selected for the next generation. assigns it a score based on how

Population:

- Genotype multiset (sample of solution space points) – evolution unit
Individual = static, does not evolve, does not adapt
Population = dynamic, evolves, adapts
- Holds set of genotypes corresponding to candidate solutions
- Selection operators are defined on populations
- Size: fixed, variable

- Diversity: number of distinct phenotypes/genotypes, number of distinct values of fitness function, entropic measures

Population:

- **Initial** -> built by randomly (uniform distribution) choosing points in the solution space and encoding them as genotypes
- Current (generation)
- Parents (pool, mating pool) -> selected from current population
- Descendants (children, offspring) -> computed from the parents pool
- **Next generation** -> selected from current population and offspring

The population is a group of candidate solutions (individuals). The first population is created randomly and contains multiple individuals, each representing a possible solution. Each individual is then tested using the evaluation function. The best solutions get higher scores. The best individuals are selected to create the next generation.

Parent selection:

Parent selection mechanism -> applied on population

- Creates the premise to improve population quality
- Directs the evolution towards increasing population quality
- Better individuals have better chances to produce offspring
- Built on probabilistic bases
- Associates chances "proportional" with individual quality
- Gives non-zero chances to weak individuals
- Minimizes the chances of getting stuck in a local optimum point

Parent selection is the process of choosing the best individuals from the population to create the next generation. These selected individuals are called parents, and they combine, via crossover and mutation, to produce offspring (new candidate solutions). This ensures that better solutions (with higher fitness) have a higher chance of passing their traits to the next generation, while maintaining diversity, so the algorithm doesn't get stuck in a local optimum.

Variation operators: must not exclude any chromosome in genotype space

Mutation:

- Unary operator
- Refreshes the genes (GA), main variation operator (ES)
- Applied (usually) on descendants produced by crossover operator
- Based on stochastic factors: application, gene selection, allele selection

- Allows modification of any gene in a chromosome with non-zero probability into any allele from the definition set

Crossover:

- Recombination – binary operator
- Main variation operator in GA, less used or not at all in other classes of EA
- Creates new genotypes derived from parents
- Based on stochastic factors
- Random choice of parents, transferred sequences
- Derived genotypes (descendants, children, offspring) may be undesirable (not viable) or quality: worse, similar, better (usually rare)

New generation selection: applied on population

- Ensures population evolution
- Participants: current population individuals and their offspring
- Size of the new generation
- **Based on individual quality, sometimes and/or additional factors: only fitness function values, only age factor, combination of fitness and age**

Termination condition: stop creating new generations -> solution is the best individual from the last generation

Direct search -> stochastic search

- Hillclimbing, simulated annealing – iterative search techniques in the solution space

Hillclimbing:

- Applied on one point (individual) – initial point randomly chosen
- At each iteration choose a point in the neighbourhood of the current point, with a better fitness (as good as possible)
- The new point becomes current point for the next iteration
- The search ends if no point in the neighborhood is better than the current point
- Depending on the starting point, it may lead to a local optimum (usually happens in multimodal problems)
- Applied repeatedly, on a large number of starting points randomly chosen (uniform distribution) from the solution space
- **Choosing neighbors depends on representation** – may influence the quality of results

Examples: change, one at a time, bits in the representation (ex. Knapsack 0-1)
 / add/subtract a "step" (ex. Function optimization) / transpositions (ex. N-queens)

```

t=0
repeat
  local=false
  randomly chose a starting point  $v_c$ 
  evaluate  $v_c$ 
  repeat
    compute neighbors of  $v_c$ 
    chose neighbor  $v_n$  with the best fitness
    if  $f(v_c) < f(v_n)$ 
      then  $v_c = v_n$ 
      else local=true
  until local == true
  t=t+1
   $V(t) = v_c$ 
until t == MAX

chose v from V so  $f(v) \geq f(V(i))$  for any  $i=1..MAX$ 

```

- N-dimensions functions

- Binary representation

- $v_c \in [a_1, b_1] \times \dots \times [a_n, b_n] \subset \mathbb{R}^n$ representation: concatenation of binary strings for each $v_c(1) \dots v_c(n)$
 - Change one bit at a time
 - Change one bit in each individual substring (all combinations)

- Real value representation

- $v_c \in [a_1, b_1] \times \dots \times [a_n, b_n] \subset \mathbb{R}^n$ is represented by the sequence of real numbers $[v_c(1) \dots v_c(n)]$
 - Change one component k : $v_c(k) = v_c(k) \pm i * \varepsilon$, $\varepsilon = 10^{-nz}$
 - Change all components (all combinations) :
 - $1 \leq k \leq n$, $v_c(k) = v_c(k) \pm i_t * \varepsilon$, $\varepsilon = 10^{-nz}$, $1 \leq t \leq nrv$
 - $v_c = (1,1) \in [-2, 4] \times [-1, 2]$, $\varepsilon = 10^{-1}$, $1 \leq i, j \leq 2$

Hillclimbing algorithm:

1. f – function to be maximized
2. MAX – number of starting points
3. Local – stuck in local optimum? (true/false)
4. V – array of final points

5. v – best point computed
6. t – point counter

Course 3. Genetic algorithms – GA

- class of evolutionary algorithms
- usually used as optimization method
- follows the general EA algorithm
- specific elements for the classical (canonical) GA: representation – binary strings, selection probability proportional to quality, high crossover probability (main variation operator), low mutation probability, age-based survivors selection (age limit = 1 generation)

Representation:

- phenotype $\leftrightarrow$ genotype
- the most important and difficult step in building an evolutionary solution
- the representation must consider the problem constraints
- depends on the problem to be solved. Influences quality of the solution
- binary strings/strings of integers/strings of real numbers/permutations

Binary representation: naturally fits problems with Boolean decision variables: $x = (x_1, \dots, x_n)$, where $x_i \in \{0, 1\}$, $i = 1, \dots, n$

Binary representation is used to encode the genotype (or genetic code) of a solution. Each binary string represents a set of parameters or features of the solution. These parameters can be anything relevant to the problem being solved.

Each gene (a part of the solution) is represented as a bit (0 or 1). The full genotype (solution) is a string of bits. Once we have the binary representation of solution, genetic operations like crossover and mutation can be directly applied on these binary strings.

Crossover combines parts of 2 binary strings (parents) to create new binary strings (offspring). Example: Parent1: 101010, Parent2: 110011, Child: 101011 (first part of Parent1, second part of Parent 2).

Mutation randomly flips a bit in the string to introduce diversity. Example: Parent: 101010, Mutation: 100010 (2nd bit was flipped).

- Knapsack 0-1 problem. Maximum weight $cmax$, objects weights $cost$, objects values val .

Genotype $\Leftrightarrow$ array v with m elements:

$$v(i) = \begin{cases} 1, & \text{if object } i \text{ is selected} \\ 0, & \text{otherwise} \end{cases}, i = 1, \dots, m$$

Constraint

$$\sum_{i=1}^m cost(i) * v(i) < Cmax$$

Fitness function

$$\sum_{i=1}^m val(i) * v(i)$$

- **Example.** Gray encoding

- Hamming distance: consider $x = (x_1, x_2, \dots, x_n), y = (y_1, y_2, \dots, y_n)$
 $d_H(x, y) = |\{i, 1 \leq i \leq n \mid x(i) \neq y(i)\}|$

- ex. $a=14, b=15, c=16, \quad a \rightarrow 01110 \quad b \rightarrow 01111 \quad c \rightarrow 10000$
 $d_H(a, b) = 1 \quad d_H(b, c) = 5$

- Encoding

- consider $\mathcal{F} = \{0, 1, 2, \dots, n\}, 2^{m-1} < n \leq 2^m - 1, x \in \mathcal{F}, b_x$

- $x \rightarrow b_x \rightarrow g_x$

$$g_x(1) = b_x(1), \quad g_x(k) = b_x(k) \text{ xor } b_x(k-1), \quad k = 2, 3, \dots, m$$

- Decoding

- $g_x \rightarrow b_x \rightarrow x$

- Step 1: compute b_x standard binary representation on m bits

- $b_x(1) = g_x(1); val = b_x(1)$
- $i = 2, \dots, m$
 - if $g_x(i) == 1, val = not(val)$
 - $b_x(i) = val$

- Step 2: compute $x = \sum_{i=1}^m b_x(m+1-i) * 2^{i-1}$

Number	0	1	2	3	4	5	6	7
Std. binary	0000	0001	0010	0011	0100	0101	0110	0111
Gray	0000	0001	0011	0010	0110	0111	0101	0100

Number	8	9	10	11	12	13	14	15
Std. binary	1000	1001	1010	1011	1100	1101	1110	1111
Gray	1100	1101	1111	1110	1010	1011	1001	1000

Representation as a string of integers:

- more than 2 (discrete) values possible for genes $x = (x_1, \dots, x_n)$, where $x_i \in Z, i = 1, \dots, n$
- finite/infinite domain
- encoding and operators chosen based on natural relations between elements – ordinal/cardinal attributes
- examples: optimization functions defined on (an interval of) Z ; encoding a set of operations/options $(0, 1, \dots, n)$ – cardinal attributes

Instead of using binary digits (0s and 1s), integers can be used to represent different features or parameters of a solution. Once you have your solutions encoded as strings of integers, you can apply genetic operators like crossover and mutation to evolve the population. They work the same as they do for the binary representation.

▪ Example – ordinal attributes

An airline intends to extend its air fleet by purchasing three types of aircrafts, The available budget is 5000 units. The three types have the following features:

- Type 1: costs 100 units, has a range of 6000 km and the Traffic Collision Avoidance System (TCAS) range is 30 km;
- Type 2: costs 60 units, has a range of 4200 km and the TCAS range is 48 km;
- Type 3: costs 50 units, has a range of 2800 km and the TCAS range is 32 km.

Compute how many aircrafts from each type should be purchased such that

- the budget is not exceeded (variation: and used as much as possible)
- the average flight range is maximized
- the average TCAS range is at least 40 km.

Representation – strings of integers $x = (x_1, x_2, x_3)$

$$x_1 \in \left\{0, 1, \dots, \left\lceil \frac{5000}{100} \right\rceil = 50\right\}, \quad x_2 \in \left\{0, 1, \dots, \left\lceil \frac{5000}{60} \right\rceil = 83\right\}, \quad x_3 \in \left\{0, 1, \dots, \left\lceil \frac{5000}{50} \right\rceil = 100\right\}$$

Constrains

$$\frac{x_1 \cdot 30 + x_2 \cdot 48 + x_3 \cdot 32}{x_1 + x_2 + x_3} \geq 40, \quad x_1 + x_2 + x_3 > 0$$

Fitness function

$$f(x_1, x_2, x_3) = \frac{x_1 \cdot 6000 + x_2 \cdot 4200 + x_3 \cdot 2800}{x_1 + x_2 + x_3}$$

Representation as strings of real values:

- Optimization functions defined on continuous intervals $x = (x_1, \dots, x_n)$, where $x_i \in \mathbb{R}$, $i = 1, \dots, n$

▪ Example

$$\begin{cases} \max_{x \in D} f(x) \\ x = (x_1, x_2, \dots, x_n)^T \\ D = \prod_i [a_i, b_i] \end{cases}$$

Representing solutions as strings of real values means encoding each parameter of a solution as a real number (rather than an integer or binary value). This approach is particularly useful when dealing with continuous domains, where the values can take any number in a range. In real-value representation, each parameter is encoded as a real number (e.g., 3.14, 9.8, -1.2). Each individual solution in the population consists of a vector of real values. These values can represent different parameters of the solution.

Once solutions are represented as real values, genetic operators (crossover and mutation) are applied directly to these real-number strings.

Representation as permutations:

- Order of events $\leftrightarrow$ distinct integer values $x = (x_1, \dots, x_n)$, where $x_i \in \{1, \dots, n\}$, $x_i \neq x_j$, $i \neq j$
- Specific variation operators that yield permutations

- 2 interpretations: consider events {a, b, c, d, e, f}, permutation {2, 5, 4, 6, 3, 1}
 $x_i \leftrightarrow$ position of event i – event a is 2nd, b is 5th, c is 4th, d is 6th, e is 3rd, f is 1st
 $x_i \leftrightarrow$ event on position i – first event is b, then e, d, f, c, a
- Types of problems: event order is important (JSS, n-queens)/adjacency is important (TSP)

In permutation representation, the solution is encoded as a sequence of unique elements, usually integers, where the order matters. Each position in the sequence corresponds to a part of the solution, and the elements themselves represent the decision or item to be arranged. Each individual solution is a permutation of a set of values.

Mutation operators:

Mutation:

- Unary operator: 1 “parent” chromosome $\rightarrow$ “descendant”
- Stochastic factor: low probability pm
- Applied on each gene, with probability pm, if representation permits – the allele may be changed or not
- In average, pm (population size) * dim (chromosome size) * n genes are modified ($pm * dim$ individuals for permutations)
- Limits: 1 gene from 1 chromosome $\rightarrow$ 1 gene from each chromosome: $pm \in [1/dim * n, 1/n]$
- Specific operators for each representation: binary – negation, integers – random reset, creep mutation (flujj), real values – uniform mutation, non-uniform mutation, permutations – swap, insert, scramble, inversion

Random reset:

- Replace the allele with a random value from definition domain: uniform distribution
- Used for cardinal attributes

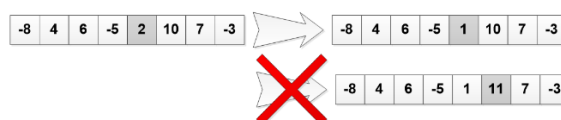
$$x_i \in \{-10, -9, \dots, 9, 10\}$$

- Example: 

Creep mutation:

- Change the allele (+-) with a small value, zero-mean distribution
- Used for cardinal attributes

$$x_i \in \{-10, -9, \dots, 9, 10\}$$



- Example:

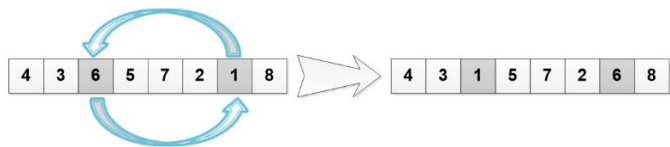
Uniform mutation (~random reset)

Non-uniform mutation (~creep mutation)

Mutation operators for permutation: applied on the entire chromosome, not on gene level

Swap:

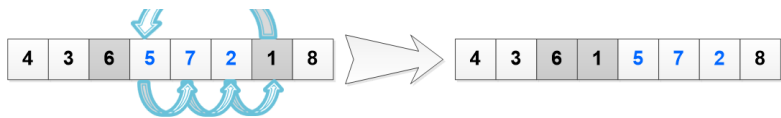
- Randomly select 2 genes $i < j$, swap x_i with x_j



- Example:

Insert:

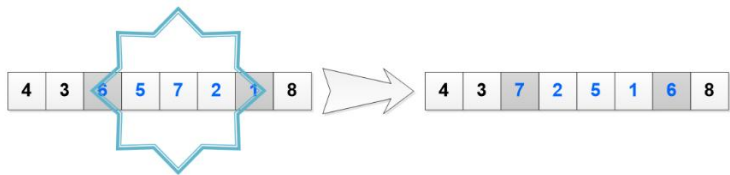
- Randomly select 2 genes $i < j$, insert x_j on position $i + 1$
- Alleles $i+1, i+2, \dots, j-1$ shift one position rightwards
- If $i+1 = j$ no change happens



- Example:

Scramble mutation:

- Randomly select 2 genes $i < j$
- Randomly scramble alleles $i, i+1, i+2, \dots, j$ in the same space



- Example:

GA – general mutation scheme, permutations

No constraints	Constraints
• O – offspring population with dim chromosomes, n genes • p_m – mutation probability • MO – mutated population	
• $MO=O;$ • for $i=1:dim$ 1. generate $r \in [0,1]$ 2. if $r \leq p_m$ 2.1. $MO(i,:)=mutate(O(i,:));$ 2.2. evaluate $MO(i,:)$	• $MO=O;$ • for $i=1:dim$ 1. generate $r \in [0,1]$ 2. if $r \leq p_m$ 2.1. $MO(i,:)=mutate(O(i,:));$ 2.2. if $viable(MO(i,:))$ • evaluate $MO(i,:)$ - else • $MO(i,:)=O(i,:)$

pace

6 8



No constraints	Constraints
• O / MO – offspring / mutated population with dim chromosomes, n genes • p_m – mutation probability	
• $MO = O$; • for $i = 1:dim$ 1. mutated=0; 2. for $j = 1:n$ 2.1. generate $r \in [0,1]$ 2.2. if $r \leq p_m$ • $MO(i,j) = \text{mutate}(O(i,j));$ • mutated=1; 3. if mutated • evaluate $MO(i,:)$	• $MO = O$; • for $i = 1:dim$ 1. mutated=0; 2. for $j = 1:n$ 2.1. generate $r \in [0,1]$ 2.2. if $r \leq p_m$ • $MO(i,j) = \text{mutate}(O(i,j));$ • mutated=1; 3. if mutate 3.1. if viable($MO(i,:)$) • evaluate $MO(i,:)$ - else • $MO(i,:) = O(i,:)$

Course 4. GA – General Recombination Scheme

- B/O – parents/descendants population
- dim – parents population size
- K – no. of recombination operations between 2 individuals from B ; each produces 2 descendants
- p_r – recombination probability

A. Problems without constraints

For $i = 1:K$

1. Randomly choose 2 individuals from B : $p_1 \neq p_2$, $p_1, p_2 \in 1, \dots, dim$
2. Generate $r \in [0,1]$

3. If $r \leq p_r$
 - 3.1. $d_1, d_2 = \text{recombine}(B(p_1, :), B(p_2, :))$
 - 3.2. **evaluate d_1 and d_2**
- else
- 3.3. $d_1, d_2 = B(p_1, :), B(p_2, :)$
4. $d_1, d_2 \rightarrow O$

B. Problems with constraints

For $i = 1:K$

1. Randomly choose 2 individuals from B: $p_1 \neq p_2, p_1, p_2 \in 1, \dots, \text{dim}$
2. Generate $r \in [0,1]$
3. If $r \leq p_r$

3.1. $d_1, d_2 = \text{recombine}(B(p_1, :), B(p_2, :))$

3.2. **for $j = 1:2$**

if $\text{viable}(d_j)$

evaluate d_j

else $d_j = B(p_j, :)$

else

3.3. asexual recombination: $d_1, d_2 = B(p_1, :), B(p_2, :)$

4. $d_1, d_2 \rightarrow O$

```

3.2. a=1
for j=1:2
    if not viable(d_j)
        a=0
    if a==1
        evaluate d_1 and d_2
    else go to 3.1

```

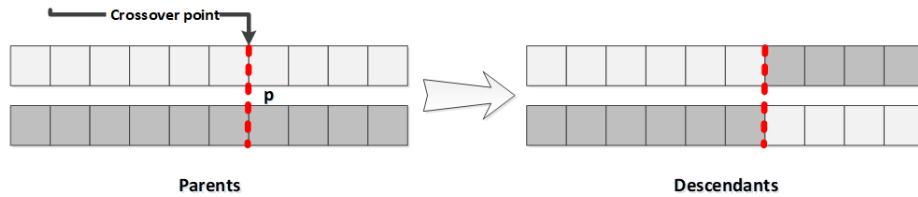
Recombination operators

Recombination (crossover): 2 parents, probability, descendants

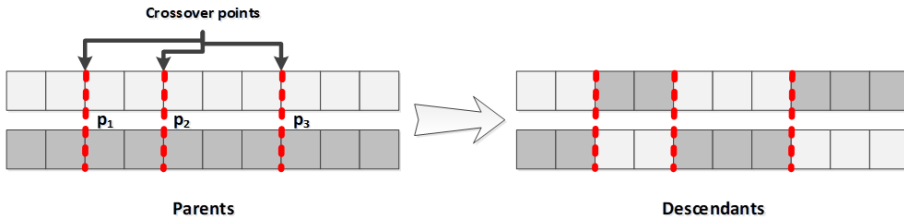
- **Binary/Z/R strings:** discrete recombination – single point (one-point)/multiple points (n-points)/uniform recombination
- **R strings:** arithmetic – simple/single/total recombination
- **Permutations** – **PMX** (Partially Mapped Crossover), **OCX** (Order Crossover), **CX** (Cycle Crossover), **ECX** (Edge Crossover)
- **Multiple recombination**

Recombination – binary strings

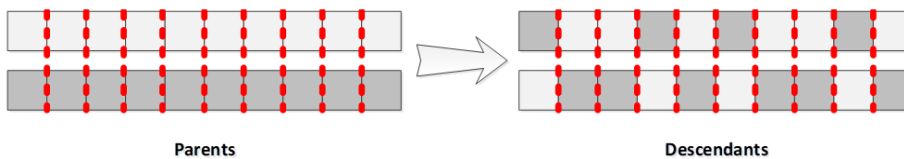
- **Single point:** p_r , one position



- **Multiple points:** p_r , n positions



- **Uniform:** p_r , every position



Recombination – integer strings

- Discrete recombination, similar to binary strings
- **Single point:** p_r , one crossover point (gene)
- **Multiple points:** p_r , n crossover points (genes)
- **Uniform:** p_r , crossover at each position (gene)
- Other operations $\leftrightarrow$ possible aberrant descendants (not integer alleles)

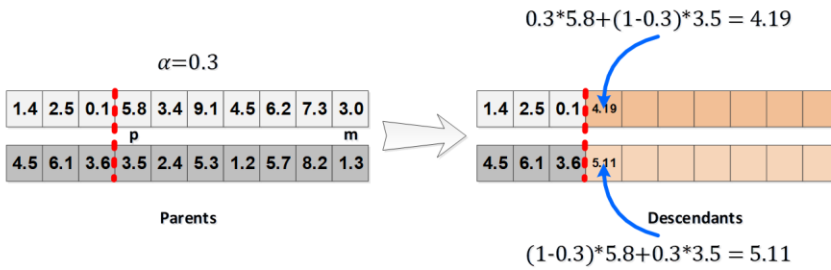
Recombination – real value strings

- Discrete recombination
- Same as binary/integer strings
- Does not produce new alleles
- Arithmetic recombination: p_r , α (weight)
- Weighted average of a gene pair: $x_1, x_2 \rightarrow y_1, y_2$

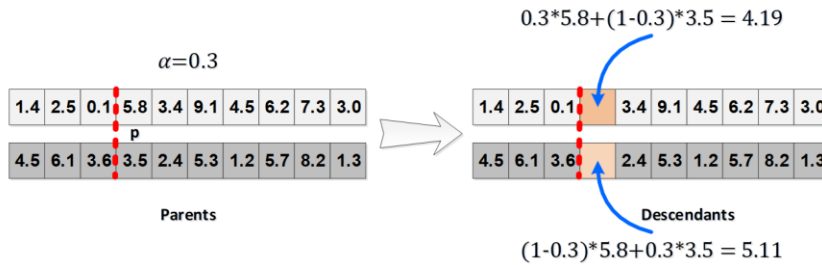
$$y_1(p) = \alpha \cdot x_1(p) + (1 - \alpha) \cdot x_2(p)$$

$$y_2(p) = (1 - \alpha) \cdot x_1(p) + \alpha \cdot x_2(p)$$

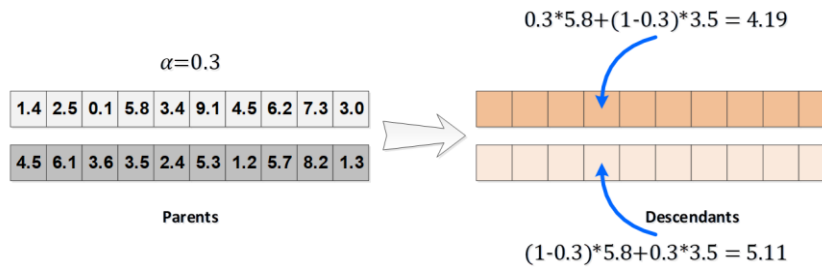
- **Simple:** from p to last gene (m)



- **Single:** only on gene p



- **Total:** on each gene



OCX

- Problems where the permutation indicates the order of events



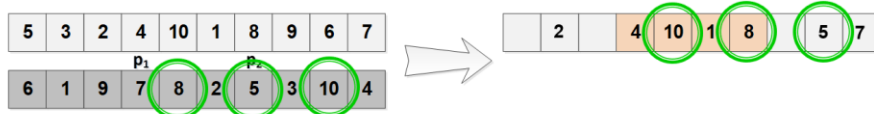
```

d( p(1):p(2) ) = x(p(1):p(2))
where = p(2)+1
for i = [p(2):m 1:p(2)-1]
  if y(i) ∉ d
    if where > m
      where=1
    d(where) = y(i)
    where = where+1

```

PMX

- Problems with adjacency dependencies



x, y – parents, d – descendant, p(1), p(2) – positions

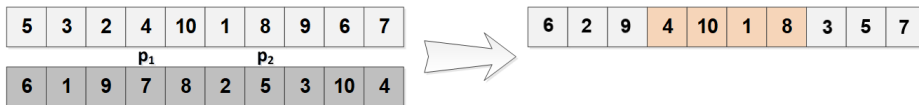
Descendant

```

d( p(1):p(2) ) = x( p(1):p(2) )
for k = p(1):p(2)
    if y(k) ∉ d
        i = position of d(k) in y
        while d(i) ≠ 0
            i = position of d(i) in y
        d(i)=y(k)
for i=1:m
    if y(i) ∉ d
        j = first free position in d
        d(j)=y(i)
    
```

```

k=7 y(k)=5 ∉ d
d(k)=8
i=5
d(i)=10
i=9
d(9)=0
d(9)=y(7)
    
```



x, y – parents, d – descendant, p(1), p(2) – positions

Descendant

```

d( p(1):p(2) ) = x( p(1):p(2) )
for k = p(1):p(2)
    if y(k) ∉ d
        i = position of d(k) in y
        while d(i) ≠ 0
            i = position of d(i) in y
        d(i)=y(k)
for i=1:m
    if y(i) ∉ d
        j = first free position in d
        d(j)=y(i)
    
```

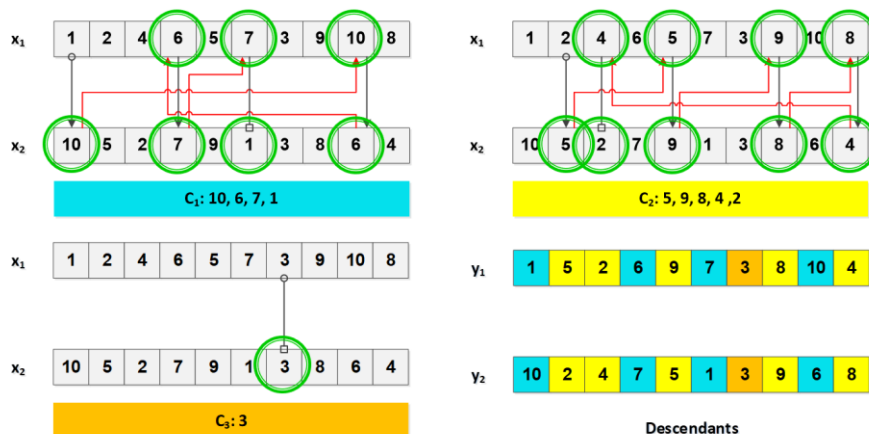
```

k=7 y(k)=5 ∉ d
d(k)=8
i=5
d(i)=10
i=9
d(9)=0
d(9)=y(7)
    
```

- Parents: 6, 9, 3

CX

- Saves as much as possible the information regarding the absolute position of an allele in parent chromosomes
- Cycle: alleles in the same gene in the 2 parents are part of the same cycle
- 2 steps: compute cycles; build descendants, by alternatively copying alleles from cycles



ECX

- Creates descendants by copying pairs of consecutive alleles that are present in at least one parents
- Saves common sequences from parents
- First step: create lists of adjacent alleles (edge table)

Multiple recombination <-> more than 2 parents (n-tuples)

- Easy to implement, but no biological equivalent
- Generally, it does not lead to better results
- **Types of multiple recombination operators:**
 1. **Based on allele frequency** – generalization of uniform crossover
 2. **Based on segmentation and combination of segments** – generalization of multiple-point crossover
 3. **Based on arithmetical operations on the alleles** – centroid type, generalization of arithmetical crossover

Course 5. Selection operators

- Parents selection process
Compute selection probability distribution:
- **FPS (Fitness Proportional Selection)**
- **Rank based selection** – linear, non-linear
- Implementation of selection mechanism: **roulette** (simple, one arm), **SUS** (multiple arm roulette), **tournament**

FPS principle:

Compute FPS selection probability distribution:

- Chromosome quality/fitness $x_i : f_i = f(x_i), 1 \leq i \leq \dim$

$$p_i = \frac{f_i}{\sum_{j=1}^{\dim} f(x_j)}$$

- Selection probability for x_i :
- In average, each individual is selected $nx_i = f_i / f$ times

Drawbacks:

- Premature convergence <-> favors highly superior individuals
- In case of comparable fitness values <-> no selection pressure
- Comparable to uniform distribution, does not lead to population quality improvement
- Different behavior on translated/transported fitness functions

FPS variants:

- FPS variants $\Leftrightarrow$ eliminate / reduce drawbacks
- By reducing the amount of information that includes them

- Windows for the current population $\mathcal{P}^t$
- Adjust fitness values (only for the selection process) $g(x) \leftarrow f(x) - \beta_p^t$
- β_p^t - adjusting factor, computed from recent evolution history (last p generations)
- Usual choice: moving average of the weakest individual $\beta_p^t = \frac{1}{p} \sum_{u=t-p+1}^t \min_{x \in \mathcal{P}^u} f(x)$
- $g(x)$ is used instead of $f(x)$ in the FPS mechanism

Sigma-scaling for current population $\mathcal{P}^t$

- Selection average $\bar{f}_t$
- Selection standard deviation $\sigma_{f,t}$

$$\bar{f}_t = \frac{1}{|\mathcal{P}^t|} \sum_{x \in \mathcal{P}^t} f(x)$$

$$\sigma_{f,t}^2 = \frac{1}{|\mathcal{P}^t| - 1} \sum_{x \in \mathcal{P}^t} (f(x) - \bar{f}_t)^2$$

$$g(x) = \max(f(x) - (\bar{f}_t - c \cdot \sigma_{f,t}), 0)$$

- c is a preset parameter (usually $c = 2$)
- $g(x)$ is used instead of $f(x)$ in the FPS mechanism

Ranks – linear:

- Uses chromosome rankings instead of $f(x)$
- Hypothesis: an average individual should have 1 chance to reproduce
- Sort population ascending on $f(x)$ values
- Allocate probabilities based on position in sorted population
- **Linear ranks** $P_{rang_l}(x_i) = \frac{2-s}{dim} + \frac{2 \cdot i \cdot (s-1)}{dim \cdot (dim-1)}$ $1 < s \leq 2$
- $P_{rang_l}(x_i)$ is used in FPS mechanism instead of $f(x_i)$ $P_{rang}(x_i) = \frac{P_{rang_l}(x_i)}{\sum_{j=1}^{dim} P_{rang_l}(x_j)}$
- For an average individual x_i : $s = 2, i = \frac{dim}{2} \Rightarrow P_{rang_l}(x_i) = \frac{1}{dim-1}$

Ranks – non-linear:

- To avoid scaling $P_{rang_l}(x_i) = \frac{2-s}{dim} + \frac{2 \cdot i \cdot (s-1)}{dim \cdot (dim+1)}$

$$\sum_{i=1}^{dim} P_{rang_l}(x_i) = 2 - s + \frac{2(s-1)}{dim(dim+1)} \sum_{i=1}^{dim} i = 2 - s + \frac{2(s-1)}{dim(dim+1)} \frac{dim(dim+1)}{2} = 2 - s + s - 1 = 1$$

- **Non-linear ranks (exponential)** $P_{rang_e}(x_i) = \frac{1 - \exp(-i)}{c}$
- Favors higher quality individuals (more powerful selection pressure)
- c is chosen such that $\sum_{i=1}^{dim} P_{rang_e}(x_i) = 1$

Example – ranks:

- Maximize $f(x) = x^2$, with $x \in \{0,1,2, \dots, 31\}$

Nr. crt.	Binary repr.	fitness	FPS	Rank l. s=1.5	Rank exp.
1	0 1 1 0 0	144	0.0598	0.1200	0.1430
2	1 0 0 1 0	324	0.1344	0.1600	0.1955
3	1 0 1 1 0	484	0.2008	0.2000	0.2149
4	1 1 0 1 1	729	0.3025	0.2400	0.2220
5	1 1 0 1 1	729	0.3025	0.2800	0.2246

- Maximize $f(x) = x^2 + 1000$, with $x \in \{0,1,2, \dots, 31\}$

Nr. crt.	Binary repr.	fitness	FPS	Rank l. s=1.5	Rank exp.
1	0 0 1 1 0	1036	0.1608	0.1200	0.1430
2	0 1 1 0 0	1144	0.1776	0.1600	0.1955
3	0 1 1 1 0	1196	0.1857	0.2000	0.2149
4	1 0 1 0 1	1441	0.2237	0.2400	0.2220
5	1 1 0 0 1	1625	0.2523	0.2800	0.2246

Simple (one arm) roulette mechanism:

- Ideal selection probability vs. Sample - Ideal vs. Concret
- Roulette** $\Leftrightarrow$ a sample of the probability distribution
- FPS type or rank based
- Algorithm** – rotate the roulette arm dim times
- Step 1.** for each chromosome x_i , $1 \leq i \leq dim$
- Evaluate quality $f_i = f(x_i)$
- Compute selection probabilities p_i and cumulated probabilities $q_i = \sum_{j=1}^i p_j$

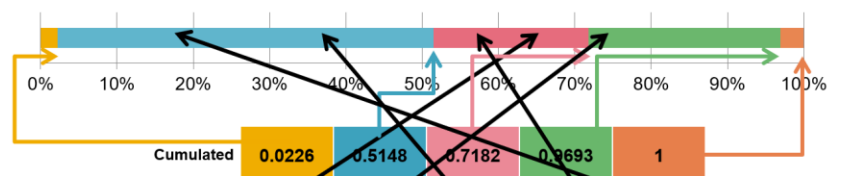
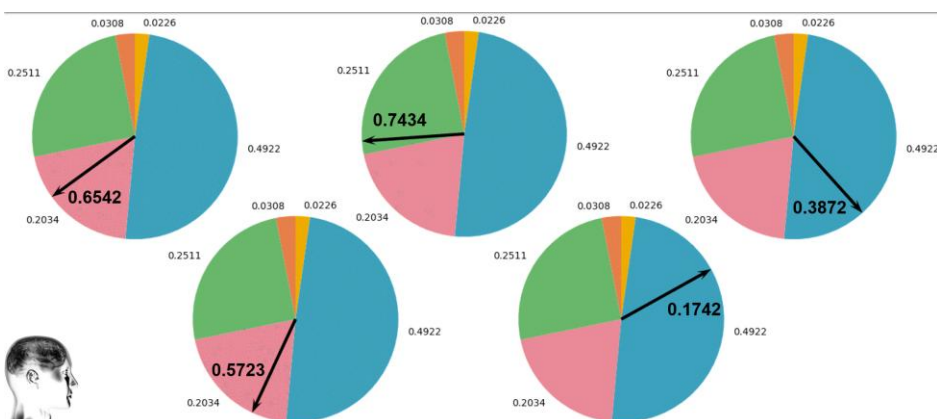
Step 2. For $k = 1..dim$

2.1. generate a random value $r \in [0,1]$

2.2. if $r \leq q_1$ parent(k) = x_1

else, parent(k) = x_i , $1 \leq i \leq dim$, for which $q_{i-1} < r \leq q_i$

Example – roulette:



SUS mechanism:

- **SUS – Stochastic Universal Sampling**
- Closer to the selection probability distribution
- Rotate only once the multiple arm roulette (dim arms, equally spaced)

Algorithm:

Step 1. For each chromosome x_i , $1 \leq i \leq dim$

- Evaluate quality $f_i = f(x_i)$
- Compute selection probabilities p_i and cumulated probabilities $q_i = \sum_{j=1}^i p_j$

Step 2. generate a random value $r \in [0, 1/dim]$; $i=1$; $k=1$; r = position of current arm; i = individual index in current population – identifies roulette sector; k = selected parent index

Step 3. while $k \leq dim$

3.1. while $r \leq q_i$

3.1.1. $parent(k) = x_i$

3.1.2. $r = r + 1/dim$; r = position of next arm

3.1.3. $k = k + 1$;

3.2. $i = i + 1$

- Each chromosome x_i will be selected at least $[dim \cdot p_i]$ times and at most $[dim \cdot p_i] + 1$ times

SUS is a fitness-proportional selection method, meaning individuals are selected with a probability proportional to their fitness score. Instead of picking individuals one by one (like in roulette wheel selection), SUS selects multiple individuals simultaneously, ensuring a more evenly distributed selection.

In Stochastic Universal Sampling, the number of pointers (or selection points) is equal to the number of individuals to be selected. This ensures that we pick exactly N individuals in a single pass, maintaining a proportional selection process while ensuring fairness.

Tournament mechanism:

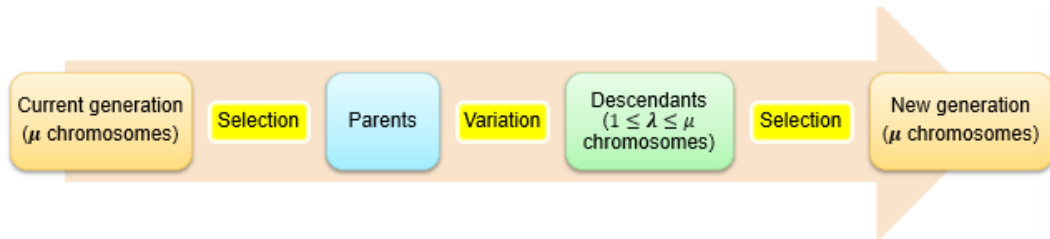
- Does not need global information
- Used when the population is too large or distributed in parallel systems
- FPS is difficult to compute
- Tournament size: $k \ll dim$ randomly chosen individuals
- Determinist or stochastic winner

- Does not follow the probability distribution (*dim* iterations)
- Advantages / disadvantages, participation limits

Algorithm

- for $i = 1..dim$
- Randomly choose k individuals from the population as set S
- find y such that $f(y) = \max_{x \in S} f(x)$ (best individual in S)
- $parent(i) = y$

GA – population models:



- Generational model: based on chromosome age
- Steady state model: based on chromosome quality

Generational model:

- Survivors selection is based on chromosome age
- Age limit (no. of generations) for individuals, no matter the fitness
- Used in simple algorithms
- $\lambda \leq \mu \Leftrightarrow$ partial replacement of current generation
- $\lambda/\mu \Leftrightarrow$ replacement rate

Extremes:

- Complete replacement, every epoch (age limit=1): $\lambda = \mu$
- Replace only 1 chromosome: $\lambda = 1$, FIFO organization of the population
- the “oldest”
- alternative: randomly select one individual to be replaced (performance varies wildly)

Steady-state model:

- Survivors selection is based on fitness
- Same mechanisms used for parents selection
- Applied on all individuals of current population + their descendants
- FPS / Ranks + roulette / SUS; tournament
- Most used: FPS/ranks + SUS, tournament

GENITOR:

- Replaces k weakest individuals in current population with k descendants (random / best)
 - **Advantage:** increases population quality (average)
 - **Disadvantage:** premature convergence $\Rightarrow$ applied on large populations / no duplicates
- Elitism:**
- Used as a supplement to other mechanisms
 - Prevents losing the best individual

Superelitism $\Leftrightarrow$ saves the best μ individuals from the $2 * \mu$ candidates

Course 6 (part 1) – Balanced transportation problem

Problem statement – Known data:

- Offer (sources, producers, suppliers): vector $O \in \mathbb{N}^m$ – m sources (producers)
- Demand (destination, consumers): vector $C \in \mathbb{N}^n$ – n destinations (consumers)
- Distribution (transport) costs: matrix $Cost_{m \times n}$

$Cost = (co_{ij}), co_{ij} \in \mathbb{R}_+$ (line $\Leftrightarrow$ offer, column $\Leftrightarrow$ demand)

- co_{ij} = cost of transporting one unit from source i to consumer j

Request

- Compute **a transport plan** – matrix $m \times n, X = (x_{ij}) \in \mathbb{N}_{m \times n}$ such that
 - Entire offer is consumed (on each line)

$$(1) \sum_{j=1}^n x_{i,j} = o_i, \quad i = 1, \dots, m$$

- Entire demand is satisfied (on each column)

$$(2) \sum_{i=1}^m x_{i,j} = c_j, \quad j = 1, \dots, n$$

- Total transportation cost is minimized

$$(3) \sum_{i=1}^m \sum_{j=1}^n co_{i,j} \times x_{i,j}$$

Example

Offer:	15 20 15	T_oferta3.txt	Costs:	T_costuri3.txt
			51 62 35 45 56	
Demand:	11 12 9 10 8	T_cerere3.txt	59 68 50 39 46	
			49 56 53 51 37	

Call: `s,c=GA_Transport('T_oferta3.txt', 'T_cerere3.txt', 'T_costuri3.txt', 10, 50, 0.8, 0.1)`

```

Best cost found: 2316.0
Transport solution:
[[ 6.  0.  9.  0.  0.]
 [ 2.  0.  0. 10.  8.]
 [ 3. 12.  0.  0.  0.]]
Remaining offer: [0. 0. 0.]
Entire supply is consumed
Remaining demand: [0. 0. 0. 0. 0.]
Entire demand is satisfied

```

GA – Solving a problem

Problem → Representation and fitness function → Generate initial population → Current generation
 → Parent pool → Descendants (recombination) → Descendants (mutation) → New generation →
 Current generation → ... (the cycle repeats) → New generation → Final population → Result →
 Decoding → Solution

Balanced transportation problem

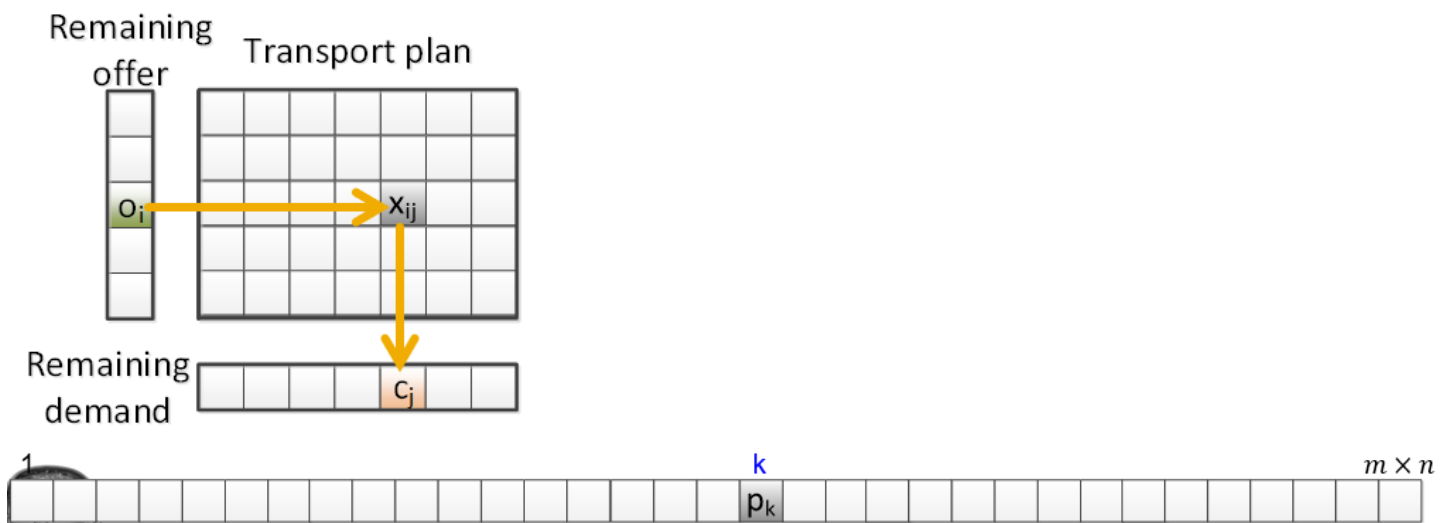
Representation – encoding

- The transport plan is built by allocating, one at a time, resources $x_{i,j}$ = how much is transported from source i to destination j
- Each assignment partially uses resources from source i and partially satisfies demand j ⇒ remaining offer, remaining demand

$$x_{i,j} = \min(o_i, c_j)$$

assign maximum possible from source i to destination j

Allocation order: one element at a time ⇒ **permutation = chromosome**



Position k in allocation order ⇒ allocate element $x_{i,j}$, where $p_k = (i - 1) \times n + j$

Any permutation is an acceptable chromosome $\Rightarrow$ **un-constrained problem**

Example

$m = 4$ sources, $n = 5$ destinations

Chromosome $\Rightarrow$ allocate resource in order:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
9	5	14	2	15	10	18	3	20	6	13	1	7	8	17	16	19	4	12	11

9 $\Rightarrow$ allocate $x_{2,4}$
 5 $\Rightarrow$ allocate $x_{1,5}$
 14 $\Rightarrow$ allocate $x_{3,4}$
 2 $\Rightarrow$ allocate $x_{1,2}$
 15 $\Rightarrow$ allocate $x_{3,5}$
 10 $\Rightarrow$ allocate $x_{2,5}$
 ...
 12 $\Rightarrow$ allocate $x_{3,2}$
 11 $\Rightarrow$ allocate $x_{3,1}$



Transport plan					
	1	2	3	4	5
1					
2					
3					
4					

Representation – decoding

```
lin,col=numpy.unravel_index(int(permutare[i]),(m,n))
```

- Build a **transport plan** from a chromosome
- Initialize
 - OR=total offer, o_r=offer vector, c_r=demand vector, k=1, x=zeros(m, n)
- While OR>0
 - Process current element (p_k) from the chromosome (permutation) $\Rightarrow$ $lin = \left\lfloor \frac{p_k}{n} \right\rfloor$, $col = mod(p_k - 1, n) + 1$
 - allocate $x(lin,col) = \min(o_r(lin), c_r(col))$
 - update $o_r(lin) = o_r(lin) - x(lin,col)$ $c_r(col) = c_r(col) - x(lin,col)$
 - update total remaining offer $OR = OR - x(lin,col)$
 - $k = k + 1$

Fitness function

- Requires chromosome decoding to compute the cost $\Rightarrow$ transport plan X
- Then compute

$$\text{cost}(X) = \sum_{i=1}^m \sum_{j=1}^n c_{i,j} \times x_{i,j}$$

- cost = minimum function, so GA uses

$$f_{\text{objectiv}}(X) = \frac{1000}{\text{cost}(X)}$$

Population model

- Constant size, generational model; no constraints $\Rightarrow$ randomly generated initial population

Termination condition

- limited number of generations + genetic diversity (at least 2 different fitness values)

Operators

- Parents selection: SUS with FPS (standard / sigma scaling)
 - FPS with sigma scaling
 - may lead to difficulties for small sized problems (all individuals have the same fitness)
 - yields better results for large size problems
- Recombination: CX
 - no constraints
- Mutation: swap
 - no constraints
- Next generation selection: age limit 1+ elitist supplement

Result

- extract the best chromosome from the final population and decode $\Rightarrow$ best computed transport plan

Course 6 (part 2) – Collinear points

Problem statement

- Consider n points in the bidimensional plan; each point A is identified by coordinates (x_A, y_A) . Use a genetic algorithm to find (if they exist) 3 collinear points among the given ones. Points A, B, C are collinear if the sum of the length of two segments is equal to the length of the third. The length of segment AB is

$$|AB| = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

- A, B, C are collinear if

$$|AB| = |AC| + |CB|$$

or

$$|AC| = |AB| + |BC|$$

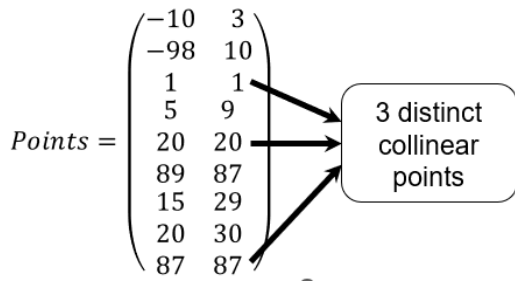
or

$$|CB| = |CA| + |AB|$$

Solving

Solution space

- **Points** – the set of given points, $\text{card}(\text{Points}) = n$, $A = (x_A, y_A) \in \text{Points}$
- $S = \text{Points} \times \text{Points} \times \text{Points}$
- Phenotype: $x = (A, B, C) \in S$, where $A \in \text{Points}$, $B \in \text{Points}$, $C \in \text{Points}$, A, B, C different!



$$\text{Points} = \begin{pmatrix} -10 & 3 \\ -98 & 10 \\ 1 & 1 \\ 5 & 9 \\ 20 & 20 \\ 89 & 87 \\ 15 & 29 \\ 20 & 30 \\ 87 & 87 \end{pmatrix} \Leftrightarrow P = \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{pmatrix} \quad \begin{cases} A = (20, 20) = P[4] \\ B = (-98, 10) = P[2] \\ C = (89, 87) = P[5] \end{cases} \Leftrightarrow x = (4, 2, 5)$$

Representation

- Genotype: $x = (a, b, c) \in A_n^3$, where $a, b, c \in P = \{0, 1, \dots, n-1\}$, a, b, c different!
- Population: matrix with integer elements, fitness values on last column
- Genotype space $S_G = \{0, 1, \dots, n-1\} \times \{0, 1, \dots, n-1\} \times \{0, 1, \dots, n-1\} = P^3$

Constraints

- A chromosome cannot include the same point more than once

Fitness

- $x = (a, b, c) \Leftrightarrow \mathbf{A} = \text{Points}[a], \mathbf{B} = \text{Points}[b], \mathbf{C} = \text{Points}[c]$
- $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are collinear if

$$|AB| = |AC| + |CB| \Leftrightarrow |AC| + |CB| - |AB| = 0 \quad \text{or}$$

$$|AC| = |AB| + |BC| \Leftrightarrow |AB| + |BC| - |AC| = 0 \quad \text{or}$$

$$|CB| = |CA| + |AB| \Leftrightarrow |CA| + |AB| - |CB| = 0 \quad \text{or}$$

- The closer one of the differences is to 0, the closer the points are to being collinear → „better“ chromosome = the minimum of the three differences is closer to 0. Optimal value is **0**.
- Function to be maximized: $f_{\text{objectiv}} = \frac{1}{1 + \text{minimum difference}}$
- Optimal value: **1**

Population model

- Generational, constant size

Initial population

- Random, from solution space, subject to restrictions: **100** chromosomes

Termination conditions

- Limited number of generations: **100**
- Genetic diversity: at least **2** different fitness values in current population
- Fitness quality: close enough to **1**. Acceptable gap: 10^{-4}

Parents selection

- SUS with FPS with sigma-scaling

Recombination

- Recombination scheme for constrained problems, representation with integer numbers
- Uniform recombination operator, probability **0,8**
- Unfeasible descendants replaced by parents, asexual reproduction

Mutation

- Mutation scheme for constrained problems, representation with integer numbers
- Cardinal attributes $\Rightarrow$ random reset operator, probability **0,1**
- Unfeasible mutated individuals replaced by original descendants

Next generation selection

- Generational model $\Rightarrow$ age limit **1**, elitist supplement

Course 7 – Job shop scheduling

GA – Planning activities in a command based production system (JSS)

Problem statement

- J – set of n jobs, $J = \{J_1, J_2, \dots, J_n\}$
- O – set of individual activities (operations)
 - each job J_i , $1 \leq i \leq n \Leftrightarrow$ ordered sequence of m activities (operations), m independent of i :

$$J_i = \{o_i^1, o_i^2, \dots, o_i^m\}, o_i^j \in O, 1 \leq j \leq m$$

$$|O| = m * n$$

- M – set of m machines that execute activities from O
 - each activity in each job J_i is executed on one machine from M
 - for each job: each activity is performed on a different machine
- $d: O \rightarrow \mathbb{R}$ function that associates a duration to each activity on its designated machine
 - Duration of each activity is preset, does not depend on the plan.
- $\text{Pred} \subseteq O \times O$, an order relation $\Leftrightarrow (x, y) \in \text{Pred} \Rightarrow y$ can be scheduled after the completion of x
 - Independent precedence order for each job

Working assumptions

- each job is defined as an ordered set of activities, order is preset.
 - $\text{Pred} \subseteq O \times O$, an order relation $\Leftrightarrow (x, y) \in \text{Pred} \Rightarrow y$ can be scheduled after the completion of x
- once started, activities can not be interrupted
- at each moment in time, any machine can only perform one activity

Scheduling an operation $o \Leftrightarrow$ assign an execution window start time for o on the designed machine

Schedule (plan) $\Leftrightarrow$ collection of activity start time assignments that includes each activity at most once

Request: find a plan that is

- **complete:** includes all activities defined in set O

- **correct:** satisfies all precedence relations
- **optimal:** total duration is minimized

Example of inputs		Jobs	Operations (for each job) designated Machines Duration for each operation
▪ n=5 jobs and m=7 machines ▪ $J_1 = \{o_1^1, o_1^2, o_1^3, o_1^4, o_1^5, o_1^6, o_1^7\}$ ▪ $J_2 = \{o_2^1, o_2^2, o_2^3, o_2^4, o_2^5, o_2^6, o_2^7\}$, ▪ ... ▪ $J_5 = \{o_5^1, o_5^2, o_5^3, o_5^4, o_5^5, o_5^6, o_5^7\}$ ▪ Operations can be sequentially numbered ▪ $J_1 = [1, 2, 3, 4, 5, 6, 7 = 7 \times 1]$ ▪ $J_2 = [8, 9, 10, 11, 12, 13, 14 = 7 \times 2]$ ▪ ... ▪ $J_5 = [29, 30, 31, 32, 33, 34, 35 = 7 \times 5]$		J_1	$[1, 2, 3, 4, 5, 6, 7]$ $[2, 1, 4, 6, 5, 3, 7]$ $[4, 11, 6, 5, 7, 2, 3]$
		J_2	$[8, 9, 10, 11, 12, 13, 14]$ $[4, 1, 2, 3, 7, 6, 5]$ $[2, 9, 3, 12, 3, 4, 5]$
		J_3	$[15, 16, 17, 18, 19, 20, 21]$ $[7, 1, 3, 4, 2, 5, 6]$ $[3, 3, 5, 9, 4, 8, 7]$
		J_4	$[22, 23, 24, 25, 26, 27, 28]$ $[1, 2, 3, 4, 5, 6, 7]$ $[3, 5, 4, 4, 7, 6, 4]$
		J_5	$[29, 30, 31, 32, 33, 34, 35]$ $[7, 5, 3, 4, 2, 1, 6]$ $[9, 4, 7, 4, 15, 11, 3]$

Representation – Encoding: phenotype $\Leftrightarrow$ genotype

A. **Genotype: a permutation of all operations (n*m genes)**

- Indicates the **order operations are analyzed for scheduling** $\Rightarrow$ **plan (phenotype)**
schedule for each operations based on an analysis performed in the order indicated by the genotype
- Genotype: a concatenation of permutations of operations on each machine
- m machines, n jobs $\Rightarrow$ m permutations of n elements each $\Rightarrow$ m*n genes
- **For each machine**, the corresponding permutation indicates the **order of analyzing operations for scheduling**
 - Assign a start moment for each operation based on
 - Precedence relations for the corresponding job
 - Machine availability
 - No-constraint problem: any genotype can be decoded into a plan

Representation – Encoding: phenotype $\Leftrightarrow$ genotype

- Consider genotype x with n*m genes (number of operations)
- For each job i operations are listed according to precedence order

$$J_i = \{o_i^1, o_i^2, \dots, o_i^m\} \Leftrightarrow \{(i-1) * m + 1, (i-1) * m + 2, \dots, (i-1) * m + m\}$$

$$J_i = \{o_i^0, o_i^1, \dots, o_i^{m-1}\} \leftrightarrow \{i * m + 0, i * m + 1, \dots, i * m + m - 1\}$$

- Each operation o_i^j , $1 \leq i \leq n$, $1 \leq j \leq m$ is identified by the number $(i - 1) * m + j$

[2, 3, 1, 5, 4]	[2, 5, 3, 2, 1]	[2, 5, 4, 1, 3]	[5, 4, 3, 2, 1]	[3, 2, 4, 1, 5]	[2, 4, 3, 5, 1]	[5, 1, 2, 4, 3]
[1, 2, 0, 4, 3]	[1, 4, 2, 3, 0]	[1, 4, 3, 0, 2]	[4, 3, 2, 1, 0]	[2, 1, 3, 0, 4]	[1, 3, 2, 4, 0]	[4, 0, 1, 3, 2]
Machine 1	Machine 2	Machine 3	Machine 4	Machine 5	Machine 6	Machine 7

Representation – Decoding: genotype $\Rightarrow$ phenotype (plan), duration

- Initialize
 - Schedule all operations at time 0 (vector of start times, initialized with 0)
 - Mark each operation as not scheduled (binary vector, initialized with 0)
- While there are unscheduled activities
 - For each job $i = 1, 2, \dots, n$
 - For each machine $j = 1, 2, \dots, m$
 - Execute Step 1 through Step 5

Step 1. Determine current operation, list of precedent operations, list of successor operations

Let $o = p_j(i)$ unscheduled; compute pr list of preceding operations for o , $succ$ list of successor operations for o :

- compute c, r (job c , position inside the job r): $o = c * nrm + r$
- if $r \neq 0$, o is part of job J_{c+1} on position r

$$pr = \{c * nrm + 1, \dots, c * nrm + r - 1\}$$

- if $r = 1$, $pr = \{\}$: first operation of job J_{c+1} does not have a predecessor

$$succ = \{c * nrm + r + 1, \dots, c * nrm + nrm - 1\}$$

- if o is multiple of m ($r = 0$), o is the last element of task J_c and

$$pr = \{(c - 1) * nrm + 1, \dots, (c - 1) * nrm + nrm - 1\}, \quad succ = \{\}$$

Step 2. Analyze prior operations

For each $op \in pr$

- if op was scheduled, start time of o must be after end time of op : if o is scheduled too early, its start time is adjusted as the end time of op : $plan(o) = plan(op) + d(op)$

Step 3. Analyze machine availability

- Needed interval: $\text{plan}(o) \rightarrow \text{plan}(o) + d(o)$; if there is another operation whose interval intersects the needed interval for o , then push the start time of o at the end time of intersection operation; repeat for each operation already planned on machine

Pas 4. Mark the operation as scheduled

Pas 5. Analyze successor operations

For each $op \in \text{succ}$

- if **op** was scheduled and its start time is before the end time of o , then cancel the scheduling for **op** (mark it as not scheduled) (wrong schedule: $\text{plan}(op) < \text{plan}(o) + d(o)$)

Fitness function

- compute total plan duration (cost) corresponding to a genotype (must be decoded first). The cost must be minimized.
- In GA implementation maximize $1/\text{cost}(p)$
 - Decode chromosome $p \rightarrow \text{plan } x \rightarrow \text{start time and end time for each operation}$
 - Highest end time $\rightarrow \text{plan duration} = \text{cost}(p)$

Initial population

- Randomly generated
 - Concatenate nr. machines permutations, each of length nr. operations
 - **Note:** representation with $\mathbb{N}$ numbers, fitness values are $\mathbb{R} \rightarrow$ store separately (population matrix + fitness vector)

Termination condition

- Limited number of generations
- Genetic diversity: at least two different fitness values in current population

Operators

- Parents selection: SUS with FPS (with sigma-scaling)
 - Selection operators do not depend on genetic representation
- Recombination: PMX

- Randomly chose one machine and apply PMX on the permutations corresponding to that machine in the two parents
- **Note:** PMX yielded better results than OCX

X						
1 2 0 4 3	1 4 2 3 0	1 4 3 0 2	4 3 2 1 0	2 1 3 0 4	1 3 2 4 0	4 0 1 3 2
0	1	2	3	4	5	6

Y						
3 2 0 4 1	1 0 4 2 3	2 4 0 3 1	3 4 0 1 2	2 1 3 0 4	2 0 1 4 3	0 4 2 1 3
0	1	2	3	4	5	6

Operators

- Mutation: insert operator
 - Randomly choose one machine and apply insert operator on the permutation corresponding to that machine in current chromosome

X						
1 2 0 4 3	1 4 2 3 0	1 4 3 0 2	4 3 2 1 0	2 1 3 0 4	1 3 2 4 0	4 0 1 3 2
0	1	2	3	4	5	6

- Selection of next generation: elitism
 - Generational model + elitism: age limit 1, prevent losing the best individual

Course 9 (part 1) – Airplane acquisition

Statement

- An air transport company wants to purchase 3 types of airplanes, within a budget of **5000** units.
- Each airplane, denoted a through c has the following features:
 - cost **100** units, autonomy **6000** km and TCAS detection range **30** km;
 - cost **60** units, autonomy **4.200** km and TCAS detection range **48** km;
 - cost **50** units, autonomy **2.800** km and TCAS detection range **32** km.
- How many airplanes of each types should be bought such that:
 - total cost **does not exceed the budget**;
 - average autonomy** is **maximum**;
 - average** TCAS detection range is at least **40** km.
 - Observation.** If the number of planes of each type is a, b and c, then

- average autonomy is $\frac{6000 \cdot a + 4200 \cdot b + 2800 \cdot c}{a+b+c}$
- average TCAS detection range is $\frac{30 \cdot a + 48 \cdot b + 32 \cdot c}{a+b+c}$

Solving

Solution space

- $S \in S_1 \times S_2 \times S_3 = [0, 50] \times [0, 83] \times [0, 100]$
- Phenotype: $x = (a, b, c) \in S$, unde $a \in [0, 50]$, $b \in [0, 83]$, $c \in [0, 100]$

Representation

- Genotype: $x = (a, b, c) \in S$, where $a \in [0, 50]$, $b \in [0, 83]$, $c \in [0, 100]$
- Population: matrix of integers, fitness on last column

Constraints

- Stay within the budget

$$100 \cdot a + 60 \cdot b + 50 \cdot c \leq 5000$$

- Average TCAS range

$$\frac{30 \cdot a + 48 \cdot b + 32 \cdot c}{a + b + c} \geq 40$$

Fitness function

- Exclusively autonomy-based

$$f_{\text{objectiv}}(x) = \frac{6000 \cdot a + 4200 \cdot b + 2800 \cdot c}{a + b + c}$$

- Include using the ~full budget

$$f_{\text{objectiv}}(x) = \frac{6000 \cdot a + 4200 \cdot b + 2800 \cdot c}{a + b + c} - \text{unused buget}$$

Initial population

- Random, from solution space, subject to restrictions: **100** chromosomes

Termination condition

- Limited number of generations: **100**
- Genetic diversity: at least **2** different fitness values in current population
- Evolution: quality increase in the last **50** generations

Parents selection

- Roulette with FPS with sigma-scaling

Recombination

- Recombination scheme for constrained problems, representation with numbers
- Uniform recombination operator, probability 0,8

Mutation

- Mutation scheme for constrained problems, representation with numbers
- Creep mutation operator, probability 0,2 limit 2

Next generation selection

- Age limit 1
- Elitist supplement

Course 9 (part 2) – Maze path

Statement

Known data:

- A labyrinth
- A starting point (entrance), a destination point (exit)

Request

- Find a path from entrance to exit (not necessarily the shortest)

Work assumptions

- Square maze, each „square“ in the maze is either an empty place, or a wall
- Each “square” in the labyrinth is a size 1 (indivisible) square
- There is an outer wall around the maze
 - There are multiple paths between any two empty squares
 - Entry: upper-left corner, exit: down-right corner (parameters)

Randomly generate a maze

- `create_maze(20, False, 0.25, True, save='labirint-3.npy')`

Representation

- Maze $n \times n$
 - Square matrix, size $2 \times n + 3$
 - 1 = wall, 3 = empty
 - Double contour wall
 - Entry: (3, 3) or any other empty point inside the maze – (2, 2) in Python
 - Exit: $(2n + 1, 2n + 1)$ or any other empty point inside the maze – $(2n, 2n)$ in Python
 - Chromosome
 - Vector with $n \times n$ elements (maximum size of a path), integers, cardinal values
 - Each element designates a “step” in the maze (movement): 0 = N, 1 = E, 2 = S, 3 = W
 - -1 = non-movement (step-in-place)
 - It is allowed to return to a previously visited square; cycles will be eliminated at the end

Fitness function

- Requires chromosome decoding $x \Rightarrow$ path in the maze, evaluate path cost
- Path evaluation rules: $\text{cost}(\text{path})$
 - How close to exit does the path end? penalize – add vertical and horizontal distance between path end and desired exit
 - Add penalization for each attempt to move „into-the-wall” (does not produce an actual movement): 1
 - Count steps: cost 1 for each move (successful or not)
 - If the path reaches the desired exit point
 - Delete remaining steps (replace with -1)
 - Eliminate cycles (sequences that return to a previously visited square)
 - Bonus for reaching the destination: -1000
- Transform into a maximum function
 - $f_{\text{objectiv}}(\text{path}) = - \text{cost}(\text{path})$

Initial population

- Randomly generates paths
 - randomly generate steps (N, E, S, W) for each gene
 - for efficiency, do not generate moves into the walls

Termination condition

- Limited number of generations
- Genetic diversity
 - At least two different fitness values in the current population

Operators

- Parents selection: SUS with linear ranks
 - Fitness values may be negative, so a rank-based distribution must be used
 - The population must be maintained sorted
- Recombination: single-point
 - no constraints
- Mutation: random reset (cardinal values)
 - no constraints
- Next generation selection: super-elitist / elitist
 - Super-elitist: gets stuck in local optima, sometimes finds a path to exit point (mainly if such a path already exists in the initial population)
 - Elitist: almost always finds a path to destination

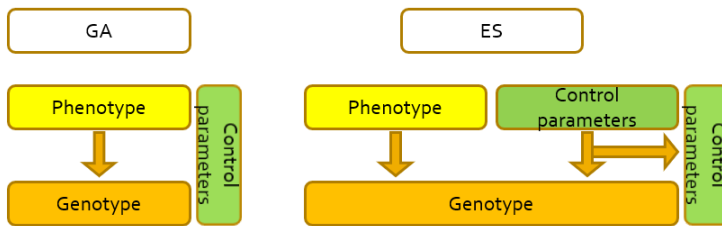
Result

- From the final population extract and decode the best individual $\Rightarrow$ best computed path
- Reaching destination is not guaranteed
 - Large size problems require more resources to have a chance to find an acceptable path
 - Longer chromosomes increase chances to find a solution
- Does not guarantee optimum solution
- If exist point is reached, fitness will be **Bonus – number of steps**

- Optional
 - Count how many chromosomes in final population reach exit point
 - All path that reach the exit point or only those with maximum fitness

Course 10 – Evolutionary strategies: Self-adaptation

- Evolutionary strategies (ES): class of evolutionary algorithms (EA)
 - Some of the control parameters are changed (adapted) during the calculations.



- Parameter self-adaptation has migrated into other types of evolutionary algorithms (EA)
- ES features
 - **Representation:** phenotype space $\Leftrightarrow$ continuous space (from $\mathbb{R}^n$)
 - genotypes $\Leftrightarrow$ *real number vectors*
 - **Parents selection:**
 - *random*, based on *uniform* probability distribution
 - **Recombination:**
 - *discrete* or intermediary (*arithmetic*)
 - **Mutation:** main variation operator
 - add gaussian noise $N(0, \sigma)$ – *gaussian perturbation*
 - σ is a self-adapting **parameter**
 - **Next generation selection:**
 - Type (μ, λ) or $(\mu + \lambda)$
- Primary algorithm: **ES2M**
 - Objective function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, for which there is $\max_{x \in \mathbb{R}^n} f(x)$
 - Compute $\tilde{x} \cong x_{max}$, $\max_{x \in \mathbb{R}^n} f(x) = f(x_{max})$

Step 1. $t = 0$; randomly chose initial point $x^t = (x_1^t, \dots, x_n^t) \in \mathbb{R}^n$

Step 2. While *termination condition* is *not* true

2.1. randomly generate z_i from normal distribution $N(0, \sigma)$, $i \in \{1, 2, \dots, n\}$

2.2. compute $y_i^t = x_i^t + z_i$, $i \in \{1, 2, \dots, n\}$

2.3. if $f(x^t) \leq f(y^t)$

$$x^{t+1} = y^t$$

else

$$x^{t+1} = x^t$$

2.4. $t = t + 1$

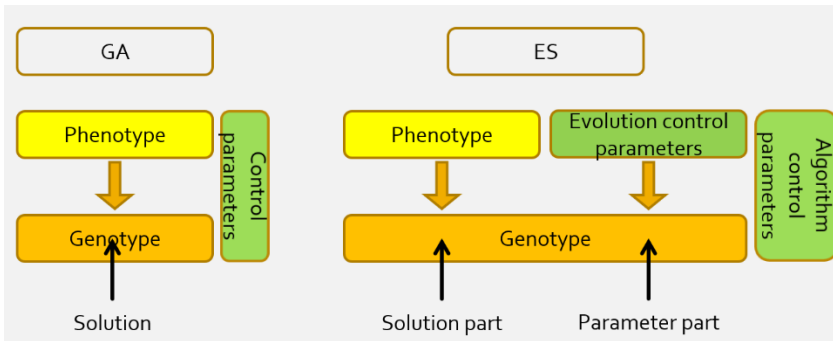
- Self-adaptation in ES2M
 - σ : mutation step size – controls mutation magnitude
 - σ changes following the $\frac{1}{5}$ success rate (success = improvement)
 - decreases if success rate is below $\frac{1}{5}$: *concentrate* the search area
 - increases if success rate is above $\frac{1}{5}$: *extend* the search area
 - adaptation happens after a fixed number of generations k
 - update rule: p_s – relative success rate, constant $c \in [0, 817, 1]$

$$\sigma = \begin{cases} \frac{\sigma}{c}, & p_s > \frac{1}{5} \\ \sigma \cdot c, & p_s < \frac{1}{5} \\ \sigma, & p_s = \frac{1}{5} \end{cases}$$

Course II – Evolutionary strategies: General algorithms

- General EA algorithms (GA, ES etc.)
 - **Statement** $\Rightarrow$ decide representation, encoding, decoding, fitness function
 - Initialize population (random), evaluate individuals
 - While termination conditions not fulfilled, repeat:
 - Select parents
 - Recombine pairs (n-tuples) of parents, evaluate descendants
 - Apply mutations on descendants, evaluate descendants

- Select next generation
- Extract best individual from last generation
- Decode extracted individual $\Rightarrow$ **solution**



- ES representation: phenotype $\leftrightarrow$ genotype
 - *Phenotype*: problem parameters $\in$ to a continuous space:

$$f(x): \mathbb{R}^n \rightarrow \mathbb{R} \rightarrow x = (x_1, \dots, x_n) \in \mathbb{R}^n$$

- *Genotype*:
 - representation of solution-candidate: similar to phenotype $\rightarrow x = (x_1, \dots, x_n) \in \mathbb{R}^n$
 - representation of ES evolution parameters: variation $\leftrightarrow$ self-adaptation
 - type σ (n_σ values – usually 1 or n): mutation step size $\rightarrow \sigma = (\sigma_1, \dots, \sigma_{n_\sigma})$
 - type α (n_α values): interactions between mutation step sizes (only correlated mutation) $\rightarrow n_\alpha = (n - n_\sigma/2) \times (n_\sigma - 1), \alpha = (\alpha_1, \dots, \alpha_{n_\alpha})$
 - general shape of a chromosome: chromosome $= (x, \sigma, \alpha)$

$$(x, \sigma) = (x_1, \dots, x_n, \sigma_1, \dots, \sigma_{n_\sigma})$$

$$(x, \sigma, \alpha) = (x_1, \dots, x_n, \sigma_1, \dots, \sigma_{n_\sigma}, \alpha_1, \dots, \alpha_{n_\alpha})$$

- Mutation in ES, general considerations
 - Non-uniform: gaussian perturbation (noise), with self-adaptation
 - $f: \mathbb{R}^n \rightarrow \mathbb{R}, x = (x_1, \dots, x_n)$, chromosome represented as (x, σ)
 - Steps:
 1. modify mutation step σ to get σ'

2. modify alleles in the solution part to get x' : for each $i \in \{1, 2, \dots, n\}$

$$x'_i = x_i + f_{x_i}$$

where f_{x_i} is randomly drawn from normal distribution $N(0, \sigma')$

- Mutation step:
 - single value: σ
 - multiple values: $\sigma = (\sigma_1, \dots, \sigma_{n_\sigma})$ (usually $n_\sigma = n$)
 - New chromosome is (x', σ')
- Mutation in ES
 - The individual is evaluated twice:
 - Direct, by computing $f(x')$ to select survivors: good if survives
 - Indirect, by the ability to create superior descendants
 - Step σ' is *better* than σ if $f(x') > f(x)$ (superior descendants)
 - Step σ' is *viable* if x' is selected in the next generation (fulfills a global quality criterium)
 - A genotype is „good“ if simultaneously x' and σ' are „better“
 - Self adaptation
 - Different mutation step values at different times and for different individuals
 - Search evolution adapts to local context, choosing the best direction
 - Personalized mutation strategy for each individual
- Mutation with **single σ parameter**
 - $(x, \sigma) = (x_1, \dots, x_n, \sigma) \rightarrow (x', \sigma') = (x'_1, \dots, x'_n, \sigma')$
 - Update rule for σ : *lognormal* distribution, *independent* update for each component
 - τ – training parameter, *fixed value*, designates *training rate* $\tau \propto \frac{1}{\sqrt{n}}$
 - Steps:
 - (1) $\sigma' = \sigma \cdot \exp(r_\tau)$ where r_τ is randomly drawn from $N(0, \tau)$
 - (2) $x'_i = x_i + f_{x_i}$ where f_{x_i} is randomly drawn from $N(0, \sigma')$, for $i = \overline{1, n}$
 - Mutation step must remain above a predefined threshold ε_σ
 - if $\sigma' < \varepsilon_\sigma$ then $\sigma' \leftarrow \varepsilon_\sigma$

- Mutation with single σ parameter
 - Algorithm:
 - for each individual in the current population (recombination descendants)
 - randomly draw r_τ from $N(0, \tau)$
 - compute $\sigma' = \sigma \cdot \exp(r_\tau)$
 - if $\sigma' < \varepsilon_\sigma$ then $\sigma' = \varepsilon_\sigma$
 - for each $i = \overline{1, n}$ (from solution part)
 - randomly draw f_{x_i} from $N(0, \sigma')$
 - $x'_i = x_i + f_{x_i}$

Low magnitude changes are preferred: $N(0, \sigma')$

- Uncorrelated mutation with **n σ parameters**
 - $(x, \sigma) = (x_1, \dots, x_n, \sigma_1, \dots, \sigma_n) \rightarrow (x', \sigma') = (x'_1, \dots, x'_n, \sigma'_1, \dots, \sigma'_n)$
 - Mutation is applied differently for each component
 - *Training parameters, fixed values:* $\tau \propto \frac{1}{\sqrt{2 \cdot \sqrt{n}}}$ $\tau' \propto \frac{1}{\sqrt{2 \cdot n}}$
 - Steps:
 - (1) $\sigma'_i = \sigma_i \cdot \exp(r_{\tau,i} + r_{\tau'})$ where $r_{\tau'} \leftarrow N(0, \tau')$ (once) and $r_{\tau,i} \leftarrow N(0, \tau)$ (individually)
 - (2) $x'_i = x_i + f_{x_i}$ where f_{x_i} is randomly drawn from $N(0, \sigma'_i)$, for $i = \overline{1, n}$
 - Mutation step must remain above a predefined threshold ε_σ
 - if $\sigma'_i < \varepsilon_\sigma$ then $\sigma'_i \leftarrow \varepsilon_\sigma$
- Uncorrelated mutation with n σ parameters
 - Algorithm:
 - for each individual in the current population (recombination descendants)
 - randomly draw $r_{\tau'}$ from $N(0, \tau')$
 - for each element $i = \overline{1, n}$ (from the parametric part)
 - randomly draw $r_{\tau,i}$ from $N(0, \tau)$
 - compute $\sigma'_i = \sigma_i \cdot \exp(r_{\tau,i} + r_{\tau'})$
 - if $\sigma'_i < \varepsilon_\sigma$ then $\sigma'_i = \varepsilon_\sigma$

- for each element $i = \overline{1, n}$ (from the solution part)

- randomly draw f_{x_i} from $N(0, \sigma'_i)$

$$x'_i = x_i + f_{x_i}$$

Parents selection in ES:

- Chromosome fitness is *not* important for ES parents selection
- The entire current population is considered the mating pool (no selection)
- For recombination the parents are randomly chosen (uniform distribution) from the current population

Recombination in ES:

- In ES each recombination operation produces ***one descendant***
 - To create λ descendants: apply recombination λ times
- Recombination types
 - Recombination of ***pairs*** of parents: 1 descendant $z \leftarrow 1 \text{ pair of parents } (x, y)$
 - discreet $z(i) = x(i) \text{ or } y(i), \text{ randomly}$
 - intermediary (convex / average) $z(i) = \frac{x(i)+y(i)}{2}$
 - ***Global*** recombination : 1 descendant $z \leftarrow n \text{ pairs of parents } (x_i, y_i) - \text{one for each gene}$
 - discreet $z(i) = x_i(i) \text{ or } y_i(i), \text{ randomly}$

intermediary (convex / average) $z(i) = \frac{x_i(i)+y_i(i)}{2}$

Notes:

- Usual recombination: global
 - Discreet for solution part: maintains genetic diversity
 - Intermediary for parameter part: cautious adaptation strategy
- Number of descendants is (much) larger than the population size
 - Contributes the maintaining the self-adaptation feature
 - Reduced selection pressure, avoids getting stuck in local optima

Next generation selection:

- Current population size - μ individuals descendants - λ individuals

- Selection: *deterministic*
 - Selection (μ, λ) – best μ individuals *from the descendants* ($\mu < \lambda$)
 - Selection $(\mu + \lambda)$ – best μ individuals *from current population and descendants*
 - Selection (μ, λ) is preferred
 - May overcome local optima
 - no descendant is identical to a current individual
 - prevents survival of weak σ individuals

Conditions for effective and functional self-adaptation:

- $\mu > 1$ – ensures the use of multiple mutation strategies
- $\lambda > \mu$ – for reduced selection pressure and to avoid premature convergence
 - usually $\lambda \approx 7 \cdot \mu$
- Use of (μ, λ) selection – ensures leaving the local optima areas
- Use intermediary recombination for parameter part